Foundations of Programming

Lab Assignment No. : 02

PROBLEM STATEMENT:

Write a menu driven program in C to implement the basic arithmetic operations such as Addition, Subtraction, Multiplication, and Division.

OBJECTIVES:

- To understand the concept of menu driven programming in C.
- To apply conditional statements and switch–case control structures.
- To perform basic arithmetic operations using user-defined input.
- To enhance logical thinking and problem-solving skills.

THEORY:

In programming, decision-making statements are used to control the flow of execution based on conditions. Two important conditional control structures are the **if–else ladder** and the **switch–case statement**. The if–else ladder is used when multiple conditions need to be checked sequentially. The program evaluates each condition one by one, and the block of code corresponding to the first true condition is executed. If none of the conditions are true, the default **else** block is executed. This structure is useful when conditions involve ranges, logical expressions, or complex comparisons.

The switch–case statement is another decision-making structure used when a variable or expression is compared against a set of constant values. Each **case** represents a possible value, and the matching case block is executed. The **break** statement is used to terminate the switch after a case is executed, preventing fall-through to the next case. A **default** case is provided to handle invalid or unmatched inputs. Switch–case is generally preferred over long if–else ladders when checking equality of a single variable because it improves readability and execution clarity.

Basic arithmetic operations form the foundation of numerical computation in programs. These operations include **addition**, **subtraction**, **multiplication**, and **division**. Addition combines two numbers to produce a sum, subtraction finds the difference between numbers, multiplication represents repeated addition, and division separates a number into equal parts. While performing division, special care must be taken to avoid **division by zero**, since dividing any number by zero is undefined and can cause runtime errors or abnormal program termination. Therefore, programs must include a condition to check that the divisor is not zero before performing division.

A **menu-driven program** is commonly used to implement arithmetic operations in an organized manner. In this approach, the user is presented with a list of choices, each representing a specific operation. Based on the user's selection, the corresponding arithmetic calculation is performed and the result is displayed. Menu-driven programs improve user interaction, clarity, and modular execution of tasks, making them widely used in beginner-level programming applications and problem-solving exercises

ALGORITHM:

Step 1: Start
Step 2: Declare variables for two numbers and choice
Step 3: Display menu:

1. Addition
2. Subtraction
3. Multiplication
4. Division

Step 4: Read user choice
Step 5:

- If choice = 1 → perform addition
- If choice = 2 → perform subtraction
- If choice = 3 → perform multiplication
- If choice = 4 →
  Check divisor ≠ 0
  If true → perform division
  Else → display error message

Step 6: Display result
Step 7: Stop

Type Program

```
#include <stdio.h>

int main() {
    float a, b, result;
    int choice;

    printf("Enter two numbers: ");
    scanf("%f %f", &a, &b);

    printf("\nMENU");
    printf("\n1. Addition");
    printf("\n2. Subtraction");
    printf("\n3. Multiplication");
    printf("\n4. Division");
    printf("\nEnter your choice: ");
    scanf("%d", &choice);

    switch (choice) {

        case 1:
            result = a + b;
            printf("Addition = %.2f", result);
            break;
```

```
}

    case 2:
      result = a - b;
      printf("Subtraction = %.2f", result);
      break;

    case 3:
      result = a * b;
      printf("Multiplication = %.2f", result);
      break;

    case 4:
      if (b != 0) {
        result = a / b;
        printf("Division = %.2f", result);
      } else {
        printf("Error! Division by zero is not allowed.");
      }
      break;

    default:
      printf("Invalid choice!");
  }

  return 0;
}
```

```c
#include <stdio.h>

int main() {
    float a, b, result;
    int choice;

    printf("Enter two numbers: ");
    scanf("%f %f", &a, &b);

    printf("\nMENU");
    printf("\n1. Addition");
    printf("\n2. Subtraction");
    printf("\n3. Multiplication");
    printf("\n4. Division");
    printf("\nEnter your choice: ");
    scanf("%d", &choice);

    switch (choice) {

        case 1:
            result = a + b;
            printf("Addition = %.2f", result);
            break;

        case 2:
```

```c
            result = a - b;
            printf("Subtraction = %.2f", result);
            break;

        case 3:
            result = a * b;
            printf("Multiplication = %.2f", result);
            break;

        case 4:
            if (b != 0) {
                result = a / b;
                printf("Division = %.2f", result);
            } else {
                printf("Error! Division by zero is not allowed.");
            }
            break;

        default:
            printf("Invalid choice!");
    }

    return 0;
```

PLATFORM:

Linux

INPUT:

Choice of operation and two numbers

OUTPUT:

Displays the result of the selected arithmetic operation

SAMPLE INPUT:

126 76

4

SAMPLE OUTPUT:

```
Enter two numbers: 126 76

MENU
1. Addition
2. Subtraction
3. Multiplication
4. Division
Enter your choice: 4
Division = 1.66
```

CONCLUSION:

Thus, the menu driven program in C successfully performs basic arithmetic operations using switch–case statements.

FAQs:

1. What is a menu driven program?

2. Why is switch–case preferred over if–else ladder in menu driven programs?

3. What is the purpose of the break statement in switch–case?

4. What is the purpose of the break statement in switch–case?

5. What are the limitations of switch–case compared to if–else?